Travaux Pratiques

Mise en œuvre de chiffrement
symétrique et asymétrique

Cryptographie et Sécurité

2025-2026

Résumé

Ce TP a pour objectif de vous familiariser avec les concepts fondamentaux de la cryptographie moderne, en particulier le chiffrement symétrique et asymétrique. Vous apprendrez à implémenter ces techniques et à comprendre leurs différences, avantages et cas d'usage.

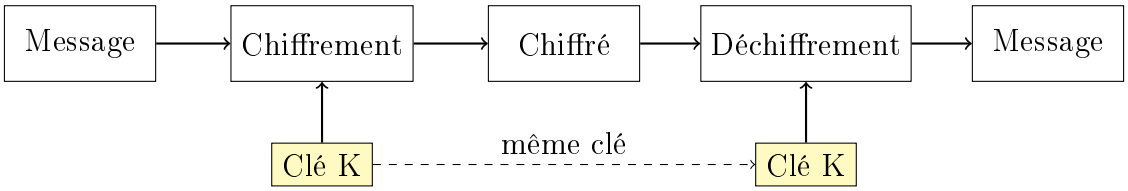
1 Objectifs du TP

À l'issue de ce TP, vous serez capable de :
- Comprendre les principes du chiffrement symétrique et asymétrique
- Implémenter des algorithmes de chiffrement (AES, RSA)
- Utiliser des bibliothèques cryptographiques (PyCryptodome, cryptography)
- Analyser la sécurité et les performances des différents algorithmes
- Mettre en place un système de communication sécurisée

2 Rappels théoriques

2.1 Chiffrement symétrique

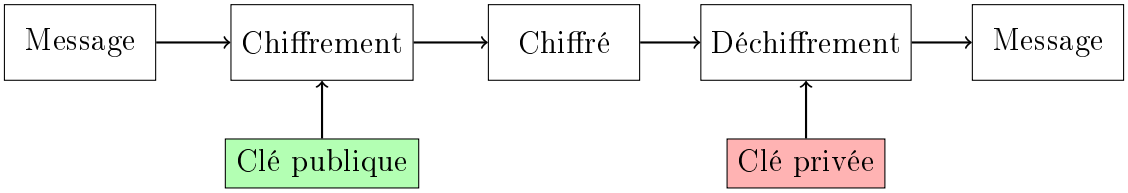
Le chiffrement symétrique utilise la **même clé** pour chiffrer et déchiffrer les données. Cette méthode est rapide et efficace pour de grandes quantités de données.



Algorithmes populaires :
- **AES** (Advanced Encryption Standard) : standard actuel, clés de 128, 192 ou 256 bits
- **DES/3DES** : obsolètes, historiques
- **ChaCha20** : alternative moderne à AES

2.2 Chiffrement asymétrique

Le chiffrement asymétrique utilise une **paire de clés** : une clé publique pour chiffrer et une clé privée pour déchiffrer. Plus lent mais permet l'échange sécurisé de clés.



Algorithmes populaires :
— **RSA** : basé sur la factorisation, clés de 2048-4096 bits
— **ECC** (Elliptic Curve Cryptography) : clés plus courtes, même sécurité
— **ElGamal** : basé sur le logarithme discret

3 Partie 1 : Chiffrement symétrique avec AES

3.1 Installation des dépendances

Installez la bibliothèque PyCryptodome :

```
pip install pycryptodome
```

3.2 Exercice 1.1 : Chiffrement AES en mode ECB

Implémentez un programme de chiffrement/déchiffrement AES en mode ECB.

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

def encrypt_aes_ecb(plaintext, key):
    """Chiffre un message avec AES en mode ECB"""
    cipher = AES.new(key, AES.MODE_ECB)
    padded_data = pad(plaintext.encode('utf-8'), AES.block_size)
    ciphertext = cipher.encrypt(padded_data)
    return ciphertext

def decrypt_aes_ecb(ciphertext, key):
    """Dechiffre un message avec AES en mode ECB"""
    cipher = AES.new(key, AES.MODE_ECB)
    padded_data = cipher.decrypt(ciphertext)
    plaintext = unpad(padded_data, AES.block_size)
    return plaintext.decode('utf-8')

# Test
key = get_random_bytes(16)  # Cle de 128 bits
message = "Message secret a chiffrer"

encrypted = encrypt_aes_ecb(message, key)
```

```
print(f"Chiffre (hex): {encrypted.hex()}")

decrypted = decrypt_aes_ecb(encrypted, key)
print(f"Dechiffre: {decrypted}")
```

Listing 1 – Chiffrement AES-ECB

Question 1.1 : Quel est le problème principal du mode ECB ? Testez avec un message contenant des répétitions.

3.3 Exercice 1.2 : Chiffrement AES en mode CBC

Le mode CBC (Cipher Block Chaining) utilise un vecteur d'initialisation (IV) et chaîne les blocs.

```
def encrypt_aes_cbc(plaintext, key):
    """Chiffre un message avec AES en mode CBC"""
    cipher = AES.new(key, AES.MODE_CBC)
    padded_data = pad(plaintext.encode('utf-8'), AES.block_size)
    ciphertext = cipher.encrypt(padded_data)
    return cipher.iv, ciphertext

def decrypt_aes_cbc(iv, ciphertext, key):
    """Dechiffre un message avec AES en mode CBC"""
    cipher = AES.new(key, AES.MODE_CBC, iv=iv)
    padded_data = cipher.decrypt(ciphertext)
    plaintext = unpad(padded_data, AES.block_size)
    return plaintext.decode('utf-8')

# Test
key = get_random_bytes(16)
message = "Message secret avec mode CBC"

iv, encrypted = encrypt_aes_cbc(message, key)
print(f"IV (hex): {iv.hex()}")
print(f"Chiffre (hex): {encrypted.hex()}")

decrypted = decrypt_aes_cbc(iv, encrypted, key)
print(f"Dechiffre: {decrypted}")
```

Listing 2 – Chiffrement AES-CBC

Question 1.2 : Pourquoi faut-il transmettre l'IV avec le message chiffré ? L'IV doit-il rester secret ?

3.4 Exercice 1.3 : Mode GCM (Authenticated Encryption)

Le mode GCM combine chiffrement et authentification.

```
def encrypt_aes_gcm(plaintext, key):
    """Chiffre avec AES-GCM (chiffrement authentifie)"""
    cipher = AES.new(key, AES.MODE_GCM)
    ciphertext, tag = cipher.encrypt_and_digest(
```

```
        plaintext.encode('utf-8')
    )
    return cipher.nonce, ciphertext, tag

def decrypt_aes_gcm(nonce, ciphertext, tag, key):
    """Dechiffre avec AES-GCM"""
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    plaintext = cipher.decrypt_and_verify(ciphertext, tag)
    return plaintext.decode('utf-8')

# Test
key = get_random_bytes(16)
message = "Message authentifie avec GCM"

nonce, encrypted, tag = encrypt_aes_gcm(message, key)
print(f"Nonce: {nonce.hex()}")
print(f"Tag: {tag.hex()}")

decrypted = decrypt_aes_gcm(nonce, encrypted, tag, key)
print(f"Dechiffre: {decrypted}")
```

Listing 3 – Chiffrement AES-GCM

Question 1.3 : Que se passe-t-il si on modifie un bit du message chiffré avant de le déchiffrer ?

4 Partie 2 : Chiffrement asymétrique avec RSA

4.1 Exercice 2.1 : Génération de clés RSA

```
from Crypto.PublicKey import RSA

def generate_rsa_keypair(key_size=2048):
    """Genere une paire de cles RSA"""
    key = RSA.generate(key_size)
    private_key = key.export_key()
    public_key = key.publickey().export_key()
    return private_key, public_key

# Generation
private_key, public_key = generate_rsa_keypair(2048)

print("Cle privee (debut):")
print(private_key[:100])
print("\nCle publique:")
print(public_key.decode('utf-8'))

# Sauvegarde dans des fichiers
with open('private_key.pem', 'wb') as f:
    f.write(private_key)

```

```
with open('public_key.pem', 'wb') as f:
    f.write(public_key)
```

Listing 4 – Génération de paire de clés RSA

Question 2.1 : Quelle est la structure d'une clé RSA ? Identifiez les composants (n, e, d).

4.2 Exercice 2.2 : Chiffrement et déchiffrement RSA

```
from Crypto.Cipher import PKCS1_OAEP

def encrypt_rsa(message, public_key_data):
    """Chiffre avec RSA-OAEP"""
    public_key = RSA.import_key(public_key_data)
    cipher = PKCS1_OAEP.new(public_key)
    ciphertext = cipher.encrypt(message.encode('utf-8'))
    return ciphertext

def decrypt_rsa(ciphertext, private_key_data):
    """Dechiffre avec RSA-OAEP"""
    private_key = RSA.import_key(private_key_data)
    cipher = PKCS1_OAEP.new(private_key)
    plaintext = cipher.decrypt(ciphertext)
    return plaintext.decode('utf-8')

# Test
message = "Message chiffre avec RSA"
encrypted = encrypt_rsa(message, public_key)
print(f"Chiffre (hex): {encrypted.hex()[:100]}...")

decrypted = decrypt_rsa(encrypted, private_key)
print(f"Dechiffre: {decrypted}")
```

Listing 5 – Chiffrement RSA avec OAEP

Question 2.2 : Quelle est la taille maximale d'un message que vous pouvez chiffrer avec RSA-2048 ?

4.3 Exercice 2.3 : Signature numérique

```
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

def sign_message(message, private_key_data):
    """Signe un message avec RSA"""
    private_key = RSA.import_key(private_key_data)
    h = SHA256.new(message.encode('utf-8'))
    signature = pkcs1_15.new(private_key).sign(h)
    return signature

def verify_signature(message, signature, public_key_data):
```

```
    """Verifie une signature RSA"""
    public_key = RSA.import_key(public_key_data)
    h = SHA256.new(message.encode('utf-8'))
    try:
        pkcs1_15.new(public_key).verify(h, signature)
        return True
    except (ValueError, TypeError):
        return False

# Test
message = "Document a signer"
signature = sign_message(message, private_key)
print(f"Signature (hex): {signature.hex()[:100]}...")

is_valid = verify_signature(message, signature, public_key)
print(f"Signature valide: {is_valid}")

# Test avec message modifie
is_valid = verify_signature(message + "x", signature, public_key)
print(f"Signature valide (message modifie): {is_valid}")
```

Listing 6 – Signature et vérification RSA

Question 2.3 : Quelle est la différence entre chiffrement et signature ? Quelles clés utilise-t-on dans chaque cas ?

5 Partie 3 : Système hybride

5.1 Exercice 3.1 : Chiffrement hybride RSA + AES

Implémentez un système combinant RSA (pour échanger une clé) et AES (pour chiffrer les données).

```
def hybrid_encrypt(message, public_key_data):
    """Chiffrement hybride: AES pour data, RSA pour cle"""
    # Generer une cle AES aleatoire
    aes_key = get_random_bytes(32)  # 256 bits

    # Chiffrer le message avec AES-GCM
    cipher = AES.new(aes_key, AES.MODE_GCM)
    ciphertext, tag = cipher.encrypt_and_digest(
        message.encode('utf-8')
    )

    # Chiffrer la cle AES avec RSA
    public_key = RSA.import_key(public_key_data)
    rsa_cipher = PKCS1_OAEP.new(public_key)
    encrypted_aes_key = rsa_cipher.encrypt(aes_key)

    return {
        'encrypted_key': encrypted_aes_key,
        'nonce': cipher.nonce,
```

```
        'ciphertext': ciphertext,
        'tag': tag
    }

def hybrid_decrypt(encrypted_data, private_key_data):
    """Dechiffrement hybride"""
    # Dechiffrer la cle AES avec RSA
    private_key = RSA.import_key(private_key_data)
    rsa_cipher = PKCS1_OAEP.new(private_key)
    aes_key = rsa_cipher.decrypt(encrypted_data['encrypted_key'])

    # Dechiffrer le message avec AES-GCM
    cipher = AES.new(aes_key, AES.MODE_GCM,
                     nonce=encrypted_data['nonce'])
    plaintext = cipher.decrypt_and_verify(
        encrypted_data['ciphertext'],
        encrypted_data['tag']
    )

    return plaintext.decode('utf-8')

# Test
long_message = "Message tres long " * 100
encrypted = hybrid_encrypt(long_message, public_key)
print(f"Cle chiffree: {encrypted['encrypted_key'].hex()[:50]}...")
print(f"Taille du message chiffre: {len(encrypted['ciphertext'])}")

decrypted = hybrid_decrypt(encrypted, private_key)
print(f"Dechiffrement reussi: {decrypted[:50]}...")
```

Listing 7 – Système de chiffrement hybride

Question 3.1 : Pourquoi utilise-t-on un système hybride plutôt que RSA seul ?

6 Partie 4 : Analyse et comparaison

6.1 Exercice 4.1 : Mesure des performances

```
import time

def benchmark_encryption(message, iterations=1000):
    """Compare les performances des algorithmes"""

    # Preparation
    aes_key = get_random_bytes(32)
    rsa_private, rsa_public = generate_rsa_keypair(2048)

    # Test AES
    start = time.time()
    for _ in range(iterations):
        nonce, ct, tag = encrypt_aes_gcm(message, aes_key)
```

```
    aes_time = time.time() - start

    # Test RSA (message court seulement)
    short_msg = message[:100]
    start = time.time()
    for _ in range(iterations):
        encrypted = encrypt_rsa(short_msg, rsa_public)
    rsa_time = time.time() - start

    print(f"AES-GCM: {aes_time:.4f}s pour {iterations} iterations")
    print(f"RSA: {rsa_time:.4f}s pour {iterations} iterations")
    print(f"Ratio: RSA est {rsa_time/aes_time:.1f}x plus lent")

# Test
message = "Message de test pour benchmark"
benchmark_encryption(message)
```

Listing 8 – Comparaison des performances

6.2 Exercice 4.2 : Tableau comparatif

Complétez le tableau suivant :

Critère	**Symétrique (AES)**	**Asymétrique (RSA)**
Vitesse	Rapide	Lent
Taille de clé	128-256 bits	2048-4096 bits
Échange de clés	Difficile	Facile
Usage principal	Chiffrement de données	Échange de clés, signatures
Taille max message	Illimitée	Limitée ($<$ taille clé)

7 Questions de réflexion

1. Expliquez pourquoi le mode ECB est considéré comme non sécurisé pour la plupart des applications.
2. Pourquoi doit-on utiliser un padding lors du chiffrement avec AES ? Quel est le risque de ne pas en utiliser ?
3. Dans un système hybride, pourquoi chiffre-t-on la clé symétrique avec la clé publique du destinataire et non avec sa propre clé privée ?
4. Quelle est la différence entre confidentialité et authentification ? Quels algorithmes fournissent quoi ?
5. Un attaquant intercepte un message chiffré avec AES-GCM. Quelles informations peut-il en tirer ? Peut-il le modifier sans être détecté ?
6. Pourquoi RSA nécessite-t-il des clés beaucoup plus grandes qu'AES pour un niveau de sécurité équivalent ?

8 Projet final : Messagerie sécurisée

Implémentez une application de messagerie simple avec les fonctionnalités suivantes :

— Génération de paires de clés RSA pour chaque utilisateur
— Échange de clés publiques
— Envoi de messages chiffrés avec système hybride
— Signature des messages pour authentification
— Vérification des signatures reçues
— Interface en ligne de commande

Bonus : Ajoutez la gestion de plusieurs conversations simultanées et la sauvegarde sécurisée des clés privées (avec chiffrement par mot de passe).

9 Ressources

— Documentation PyCryptodome : `https://pycryptodome.readthedocs.io`
— NIST Standards : `https://csrc.nist.gov`
— RFC 8017 - PKCS#1 RSA : `https://tools.ietf.org/html/rfc8017`
— Cours de cryptographie : Khan Academy, Coursera

Annexe : Conseils de sécurité

1. **Ne jamais** implémenter votre propre algorithme cryptographique en production
2. Toujours utiliser des bibliothèques bien testées et maintenues
3. Générer les clés et IV avec des générateurs aléatoires cryptographiques
4. Ne jamais réutiliser un nonce/IV avec la même clé
5. Utiliser des tailles de clés conformes aux recommandations actuelles
6. Privilégier le chiffrement authentifié (GCM, etc.)
7. Protéger les clés privées avec un mot de passe fort